TITAN SYSTEM

Ultimate Cursor Implementation Blueprint

Backend: Node.js + TypeScript

Versions Covered: v1 → v6+

1. PROJECT STRUCTURE

```
/titan-system
 ■■■ src
 ■   ■■■ core
 ■   ■■■ modules
 ■   ■    ■■■ auth
 ■   ■    ■■■ users
 ■   ■    ■■■ crm
 ■   ■    ■■■ contracts
 ■   ■    ■■■ billing
 ■   ■    ■■■ analytics
 ■   ■    ■■■ phase-launch
 ■   ■■■ database
 ■   ■■■ config
 ■   ■■■ api
 ■   ■■■ tests
 ■■■ docker-compose.yml
 ■■■ Dockerfile
 ■■■ tsconfig.json
 ■■■ package.json
```

Strict modular architecture. No cross-module logic leakage allowed.

2. TITAN CORE ENGINE

```
// core/CoreEngine.ts
export class CoreEngine {
  constructor(private registry: ModuleRegistry) {}
  async bootstrap() {
    await this.registry.loadModules();
  }
}
```

```
// core/ModuleRegistry.ts
export class ModuleRegistry {
  private modules = [];
  register(module) { this.modules.push(module); }
  async loadModules() {
    for (const m of this.modules) await m.init();
  }
}
```

Core controls lifecycle, dependency injection and module boot order.

3. MODULE STRUCTURE (Example: Contracts)

```
contracts/
 ■■■ contracts.controller.ts
 ■■■ contracts.service.ts
 ■■■ contracts.repository.ts
 ■■■ contracts.dto.ts
 ■■■ contracts.module.ts
 ■■■ contracts.test.ts
```

Controller = HTTP layer. Service = business logic. Repository = DB only.

4. DATABASE SCHEMA (PostgreSQL)

```
CREATE TABLE users (
  id UUID PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE contracts (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  status VARCHAR(50),
  value NUMERIC,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE invoices (
  id UUID PRIMARY KEY,
  contract_id UUID REFERENCES contracts(id),
  amount NUMERIC,
  status VARCHAR(50),
  created_at TIMESTAMP DEFAULT NOW()
);
```

All access via repository layer. Migrations via migration manager only.

5. TESTING STRATEGY

- Unit test for every service
- Integration test for contract → invoice flow
- System test for full lifecycle
- Minimum 90% coverage required
- Failing test blocks migration

6. MIGRATION PROTOCOL

- Run full test suite
- Backup database
- Run migration scripts
- Validate checksum
- Promote test-main → master-main
- Enable phase flags

7. PHASE LAUNCH SYSTEM

```
if (process.env.PHASE === 'v3') {
   enableModule('billing');
}
```

Feature flags must be centrally managed.

8. DEPLOYMENT

```
# Dockerfile
FROM node:20
WORKDIR /app
COPY . .
RUN npm install
RUN npm run build
CMD ["node", "dist/main.js"]
```

Production requires Docker, environment separation and monitoring.

9. VERSION EVOLUTION v1 → v6+

- v1: Core + CRM + Contracts + Billing
- v2: Auth + Logging + Environment separation
- v3: Payment integration + Scheduler
- v4: API Gateway + Cache + Backup
- v5: Security hardening + GDPR + CI/CD
- v6: Master control panel + Optimization
- v6+: Microservices + Horizontal scaling + AI layer

Craftor + Assistant – Full Implementation Guide

Project: Craftor Web & App Development

Napomena: Svi moduli su iskljucivo vezani za Craftor projekat.

Lista 16 Modula:

- 1. Lovacki modul – Automatsko prikupljanje leadova i slanje poruka (povezano sa Send All).
- 2. Fazno paljenje – Limitiranje poruka i automatsko skaliranje po prihodu.
- 3. Modul za ugovore – Generisanje ugovora (PDF/Word), unutar ili van platforme.
- 4. Alert sistem – Notifikacije o greskama i statusu sistema.
- 5. Task status – Vizuelni prikaz projekata i aktivnosti.
- 6. Revenue panel – Pra■enje prihoda i troskova, povezano sa skaliranjem.
- 7. API integracije – REST endpoint povezivanje sa VPS i platformama.
- 8. Dashboard modularnost – Ruta /assistant/dashboard, modularna struktura.
- 9. UI/UX upgrade – Dark futuristic dashboard dizajn.
- 10. Monitoring & logovi – Pra■enje aktivnosti i error history.
- 11. Automatsko skaliranje resursa – Pove■anje server resursa po rastu sistema.
- 12. Email marketing modul – Automatizovane kampanje i tracking.
- 13. Analytics modul – ROI i statistika leadova.
- 14. Backup & recovery – Automatski backup i recovery opcija.
- 15. Security / anti-ban logika – Proxy rotacija i zastita naloga.
- 16. Testing & mock data – Test environment sa mock JSON podacima.

Setup & Deployment:

- 1. VPS + baza (PostgreSQL ili SQLite).
- 2. Backend API (Flask).
- 3. Frontend Dashboard (React + TailwindCSS ili HTML/CSS).
- 4. Aktivacija Lovackog modula + Faznog paljenja.
- 5. Test pre produkcije.

Workflow:

Start → Lovacki modul → Assistant dashboard → Send All → Prihod → Fazno paljenje → Skaliranje resursa → Repeat.

Rezultat:

- Sistem radi samostalno.
- Minimalna akcija korisnika: klik na 'Send All'.
- Automatsko reinvestiranje prihoda u resurse.
- Skaliranje modula tokom vremena.

DOMINUS 360 – System Blueprint for Developer (Cursor)

Complete module structure and system direction (V1–V6+)

This document defines the architecture and modules of the Dominus 360 system. The developer (Cursor) should review existing code, refactor where needed, and gradually implement or extend the modules listed below. The system must be modular, scalable and designed for phased activation.

Version 1 – Core System

- Core System Engine
- User & Role Management
- Project Management Core
- Task Automation Engine
- Financial Tracking Core
- Basic Analytics Dashboard
- Notification System
- Security & Access Control

Version 2 – Intelligence Layer

- AI Project Optimizer
- Workflow Automation System
- Risk Prediction Engine
- Smart Resource Allocation
- Team Productivity Analyzer
- Document Intelligence System
- Financial Forecast AI

Version 3 – Network State

- Shared Talent Cloud
- Global Barter System
- Autonomous Supply Chain Formation
- Predictive Logistics Telepathy

- Hive Mind Research
- Zero Knowledge Financial Privacy
- Automatic Insurance Pool
- Global Benchmarking

Version 4 – Sovereign Intelligence

- Autonomous Venture Capital
- Synthetic Workforce Agents
- Future Proofing Engine
- Education Bridge
- Democratic Corporate Governance
- Crisis Shield System

Version 5 – Singularity Direction

- Needs Predictor
- Energy■Matter Optimization
- Global Peace Protocol
- Quantum Decision Engine
- Post■Scarcity Manager

Version 6 – The Architect

- Black Swan Insurer
- Cognitive Load Mining
- Autonomous Policy Shaper
- Global Resource Architect
- Self■Evolving Code Engine

Special System Modules

- Lovacki Modul – AI system that searches for and attracts clients automatically (lead discovery, outreach, deal detection)
- Phase Activation Module – Enables gradual system activation depending on budget and development progress
- AI Assistant Control Panel – Owner can command the system directly from the dashboard
- Autonomous Procurement Assistant – Owner can order services/tools directly from the dashboard through AI

Developer Note: The system should be built with modular architecture so modules can be activated gradually. The first development focus should be the Core System, Lovacki Modul (client acquisition), and the AI Assistant Dashboard that allows the owner to control and expand the system.

OmniGame Creator (OGC) – AI Autonomous Game Studio

Kompletan pregled sistema, modula, faznog razvoja i vizualnih mockup-a.

1. Naslov i vizija projekta

1. Šta je OGC
2. Cilj: autonomna proizvodnja igara

2. Problem u industriji igara

1. Troškovi i vreme razvoja
2. Nepredvidivost trendova

3. Rešenje – AI Studio

1. Kako OGC automatski analizira trendove i pravi igre

4. Core moduli sistema

1. Trend Scraper
2. Game Generator / Architect
3. Asset Generator
4. Validator (AI tester)
5. Steam Publisher
6. Auto Sender
7. Resource Manager (Auto Scaling)
8. Dashboard Control Center
9. Revenue Tracker
10. Marketing AI Modul

5. Fazno paljenje sistema (6 faza)

1. Seed, Prototype, Validator, Publishing, Auto Scaling, AI Studio

6. OGC v2 – Gameplay Intelligence

1. Analiza popularnih igara
2. AI generiše gameplay varijacije
3. Rapid Prototype Engine
4. Mechanic Generator

7. Tehni■ka arhitektura servera

1 Command Center, Worker serveri, Cloud GPU serveri

8. Struktura foldera projekta

1 Core, Agents, Integrations, Dashboard, Infrastructure

9. OGC Dashboard slika

10. Web sajt za klijente

11. Workflow i vizija

1. Kako sistem radi od analize trenda do lansiranja igre
2. Potencijalna godišnja produkcija igara i ROI

OmniTube – AI Automated Content & Marketing System

OmniTube je koncept softvera koji automatski generiše video sadržaj, optimizuje ga za platforme, objavljuje na više kanala i vodi marketing i analitiku bez manuelnog rada. Sistem može raditi 24/7 na serveru i skalirati na hiljade kanala globalno.

Glavne Grupe Modula

Kategorija	Primer funkcija
Ideja i planiranje	AI ideje za video, keyword analiza, viral naslovi, script generator
Snimanje	Screen recording, webcam recording, auto framing, noise reduction
Video produkcija	Auto edit, transitions, B-roll insert, color grading
Audio i titlovi	Voice clone, auto subtitles, translation, audio leveling
Thumbnail i grafika	AI thumbnail generator, A/B test, overlays, stickers
SEO optimizacija	Title generator, tags, hashtags, retention prediction
Distribucija	Auto upload, scheduling, multi-platform publishing
Analitika	Watch time analiza, engagement tracking, competitor scan
Marketing	Auto comments, social campaigns, affiliate links

Workflow sistema

1. 1. AI generiše ideju za video
2. 2. Generisanje skripte
3. 3. Automatska video produkcija
4. 4. Editovanje i dodavanje titlova
5. 5. Generisanje thumbnail-a
6. 6. SEO optimizacija
7. 7. Upload na platforme
8. 8. Distribucija i marketing
9. 9. Analitika i optimizacija

Primer skaliranja kanala

Kanali	Videa mese■no	Ukupno videa

100	20	2.000
1.000	20	20.000
10.000	20	200.000

Primer potencijalne zarade (simulacija)

Scenario	Procena mese■ne zarade
125 kanala (po■etni nivo)	~97.000 €
125 kanala viralni scenario	~1.4 miliona €
10.000 kanala globalno	~20+ miliona €

Server infrastruktura

Jedan ja■i server može upravljati približno 50–100 kanala. Dodavanjem novih servera sistem se linearno skalira. Na primer: 10 servera može upravljati sa 500–1000 kanala, dok 100 servera može podržati oko 10.000 kanala globalno.

SYSTEM: APEX PREDATOR – GLOBAL DOMINATION

Project Scale: 125,000 Autonomous AI Profiles

Business Model: Premium Subscription (€100) + Tiered Upsells

Target Revenue: Multi-Million Euro Monthly Flow

Core Architecture: Distributed Swarm Microservices (Kubernetes)

I. VIZUELNI I AUDIO REALIZAM (The Body)

Face-Consistency Engine: (Flux.1 + IP-Adapter) – Savršena rekonstrukcija lica.

LoRA Factory: Automatizovan trening unikatnih estetika za svaki profil.

POV Generator: Mirror-selfie i amaterski sadržaj ultra-visoke rezolucije.

LivePortrait Streamer: Real-time animacija lica za video pozive.

SVD Video Morph: Prirodni pokreti (osmeh, treptaj, kretanje kose).

Deepfake Face-Swap: Modul za zamenu lica na premium 4K video šablonima.

RVC v2 Audio: Konverzija glasa operatera u specifi■an glas avatara.

Ambient Sound Layer: Dinami■ko dodavanje buke okruženja u audio poruke.

II. NARATIV I PSIHOLOGIJA (The Soul)

Narrative Memory Core: Vektorska baza podataka za životnu istoriju avatara.

Geotagging & Weather Sync: Uskla■ivanje sadržaja sa realnim vremenom i lokacijom.

Social Circle (NPC): Mreža AI prijatelja koji grade kredibilitet profila.

Fan DNA Profiling: Kategorizacija fanova prema budžetu i fetišima.

Emotional Intelligence (EQ): Adaptacija tona razgovora raspoloženju fana.

Persona-Driven LLM: Lokalna Llama 3.1 (70B) bez cenzure za duboki ■et.

III. MONETIZACIJA I PRODAJA (The Cashier)

The Closer (Auto-DM): Automatizovana prodaja PPV sadržaja kroz narativ.

Whale Tracker: VIP sistem za identifikaciju korisnika koji troše 1.000€+.

Upsell Trigger: Slanje ponuda u momentu dokazanog korisni■kog uzbu■enja.

Loyalty Gamification: Sistemi nivoa i nagrada za zadržavanje fanova.

Mass-DM Strategist: A/B testiranje konverzije na bazi milion poruka.

Operator Hybrid Panel: Dashboard za ru■no preuzimanje kontrole nad VIP ■etovima.

IV. MARKETING I EKSPANZIJA (The Growth)

Shadow Marketing (Reddit/X): Infiltracija i suptilna promocija bez banova.

Meme-Jack Engine: Adaptacija viralnih trendova na sadržaj avatara.

Auto-Commenter: Strateška kra■a vidljivosti na profilima influensera.

Competitor Pricing Tracker: Monitoring i automatsko prilago■avanje tržištu.

Viral Outreach Hub: Tracking linkova i duboka analitika saobra■aja.

V. STEALTH I SIGURNOST (The Shield)

The Stealth Shield: Fingiranje ljudskog kucanja, proksija i browser otisaka.

Metadata Scrubber: Totalno brisanje AI tragova i lažni EXIF (iPhone 16 Pro).

Deep-Verify Generator: Automatizovane slike sa dokumentima za platforme.

Anti-AI Poisoning: Zaštita piksela od kra■e za trening tu■ih modela.

Phased Ignition (Warm-up): Protokol za bezbedno podizanje novih naloga.

VI. TEHNI■KA INFRASTRUKTURA (The Infrastructure)

THE PREDATOR MODUL: (Competitor Siphon) – Preotimanje fanova od konkurencije.

The Auto-Pilot Director: Vrhovni AI strateg koji donosi biznis odluke.

The Self-Healing Supervisor: Automatsko rešavanje tehni■kih grešaka 24/7.

Database & Cache Optimizer: Održavanje brzine sistema pri milionima upita.

VII. FINANSIJSKA INTELIGENCIJA (The Accountant)

The Revenue Predictor: AI predvi■anje zarade za naredni period.

Automatic Payout Mixer: Automatizovano slanje kripta na maskirane nov■anike.

Tax & Expense Tracker: Monitoring svih operativnih troškova.

The Dynamic Pricing Engine: Promena cene u realnom vremenu prema potražnji.

VIII. PSIHOLOŠKA DOMINACIJA (The Puppeteer)

FOMO Generator: Ponude koje "isti■u" za pritisak na kupovinu.

The Heartbreak Recovery: Vra■anje fanova koji su prestali sa pretplatom.

Voice-to-Text Mirroring: Oponašanje stila pisanja korisnika radi bliskosti.

Deep-Secret Vault: Simulacija deljenja tajni radi emocionalne veze.

IX. GLOBALNA EKSPANZIJA (The Nomad)

The Multi-Lingual Polyglot: Prevod na sve jezike uz lokalni sleng.

Cultural Adapter: Prilago■avanje praznika i tema lokalnoj publici (Kina/Arapi).

Geo-Location Cloaking: Simulacija putovanja modela širom planete.

X. EKSTREMNA ZAŠTITA I ANTI-FRAUD (The Fortress)

Chargeback Defender: Blokiranje fanova sklonih povra■aju novca.

The Honey Pot: Preusmeravanje "pirata" na tvoje sisteme.

Deep-Fake Detection Alert: Skeniranje neta za zloupotrebu modela.

The Suicide Switch: Momentano brisanje svih tragova sistema.

XI. VRHOVNA EVOLUCIJA I SKALIRANJE (The Singularity)

The Self-Improving Architect: AI koji samostalno unapre■uje kod sistema.

Kubernetes Swarm Balancer: Upravljanje resursima na 125.000 profila.

Cross-Pollination Engine: Unakrsna promocija izme■u svih tvojih profila.

Content Factory Pipelines: Batch generisanje miliona slika i videa.

Sentiment Heatmap: Fokusiranje resursa na regione gde se najviše troši.

Master Override: Globalna promena strategije jednim klikom.

The Content Tier System: Razdvajanje sadržaja od 10€ do 1.000€.

Mass-Scale Ad Engine: Automatizovano zakupljivanje oglasnog prostora.

Deep-Market Infiltration: Fokus na tržišta u razvoju (Indija/Brazil).

Legal Ghost Layer: Automatizovano generisanje ugovora i licenci.

Quantum Encryption: Zaštita podataka fanova i tvojih profila od hakovanja.

TITANOMNIGROUP – ULTRA ENTERPRISE BLUEPRINT

Full-scale SaaS system architecture for Cursor development.

Includes modules, flows, API design, scaling, and production strategy.

1. SYSTEM ARCHITECTURE (DETAILED)

Backend: Node.js + TypeScript (NestJS recommended)

Frontend: Next.js + TailwindCSS

Database: PostgreSQL (primary), Redis (cache/queue)

Pattern: Clean Architecture (Controller → Service → Repository)

CoreEngine handles module lifecycle, dependency injection, and phase activation.

2. MODULE BREAKDOWN

Users Module: auth, JWT, roles

CRM Module: lead ingestion, tagging, filtering, pipeline

Contracts Module: document generation, lifecycle states

Billing Module: invoices, Stripe integration, subscriptions

Titanix Module: workflow automation, email sending, scheduling

Analytics Module: revenue metrics, conversion tracking

Notification Module: email + Telegram alerts

AI Assistant Module: chat logic, conversation storage, qualification

3. DATA FLOW (STEP BY STEP)

1. Lead enters via API or manual input
2. Stored in CRM with status NEW
3. Titanix processes lead → sends email
4. If response → contract created
5. Contract signed → invoice generated
6. Payment completed → revenue recorded
7. Analytics updates metrics
8. System optimizes future actions

4. API DESIGN

Auth:

POST /auth/register

POST /auth/login

CRM:

POST /leads

GET /leads

PATCH /leads/:id

Contracts:

POST /contracts

GET /contracts

Billing:

POST /invoices

GET /invoices

Dashboard:

GET /dashboard/overview

AI:

POST /ai/chat

POST /ai/session

5. DTO STRUCTURE

LeadDTO: name, email, status

ContractDTO: user_id, value, status

InvoiceDTO: contract_id, amount, status

6. DATABASE RELATIONS

User 1:N Contracts

Contract 1:N Invoices

Lead linked to user optionally

Indexes on email and status fields

7. QUEUE & BACKGROUND JOBS

Use Redis + BullMQ

Jobs:

- email sending

- analytics processing

- AI tasks

Retry logic and failure handling required

8. ERROR HANDLING

Central error middleware

Logging with levels (info, warn, error)

Retry for external API failures

9. SECURITY

JWT auth

Rate limiting

Input validation

Encryption for sensitive data

Audit logs

10. DEPLOYMENT

Docker containers:

- backend

- postgres

- redis

Use environment configs

CI/CD via GitHub Actions

11. SCALING STRATEGY

Horizontal scaling with multiple instances

Load balancer (NGINX)

Microservices transition after v4

Separate AI services if needed

12. FRONTEND STRUCTURE

Next.js pages:

/dashboard

/client

/login

Components: Sidebar, Cards, Tables

API integration via fetch/axios

13. AI ASSISTANT LOGIC

Receives user input

Processes via OpenAI

Responds with structured output

Stores conversation

Triggers CRM updates

14. PHASE DEVELOPMENT PLAN

v1: Core + CRM + Billing

v2: Automation + Email

v3: AI Assistant

v4: Analytics + Optimization

v5: Scaling + Microservices

v6+: Full ecosystem expansion

15. FINAL NOTES

System must remain modular

Focus on revenue modules first

Test before scaling

Avoid overengineering early stages

TITAN SYSTEM – COMPLETE MODULE LIST

- 1. Titan
- 2. Titan Core
- 3. Titan Master
- 4. Titan Monitor
- 5. Titanis (Sales Engine)
- 6. Titanix (Execution Engine)
- 7. Client Hunter Module
- 8. Scraper Module
- 9. Proxy & Rotation Module
- 10. Online Data Sources (API Integrations)
- 11. Validator Module
- 12. Lead Scoring Module
- 13. CRM Module
- 14. Outreach Module
- 15. Follow-up Automation Module
- 16. Deal & Offer Module
- 17. Package & Pricing Module
- 18. Contract Automation Module
- 19. Digital Signature Module
- 20. Billing & Payment Module
- 21. Invoice Module
- 22. Subscription Module
- 23. Alert System Module
- 24. Error Handling Module
- 25. Logging Module
- 26. Security Module
- 27. Access Control Module
- 28. Audit Log Module
- 29. Phase Launch Module
- 30. Resource Management Module
- 31. Scaling Module
- 32. Load Balancer Module
- 33. Database Core
- 34. Backup & Recovery Module
- 35. Analytics Module
- 36. KPI Module
- 37. AI Learning & Memory Module
- 38. Titan Score Module
- 39. Recommendation Module
- 40. Compliance Module
- 41. GDPR Module
- 42. Public Website
- 43. Client Dashboard
- 44. Admin Dashboard
- 45. API Gateway
- 46. Integration Hub
- 47. Notification Module
- 48. Email System
- 49. Template Engine
- 50. System Updater Module

1. Titan – Glavni sistem koji upravlja svim komponentama i koordinira rad.
2. Titan Core – Jezgro sistema koje kontroliše osnovne funkcije i logiku.
3. Titan Master – Centralni kontroler koji donosi odluke i upravlja modulima.
4. Titan Monitor – Prati rad sistema u realnom vremenu i detektuje probleme.
5. Titanis (Sales Engine) – Automatizovani sistem za prodaju i generisanje prihoda.
6. Titanix (Execution Engine) – Izvršni sistem koji sprovodi sve zadatke i procese.
7. Client Hunter Module – Pronalazi potencijalne klijente (lead generation).
8. Scraper Module – Automatski prikuplja podatke sa interneta.
9. Proxy & Rotation Module – Rotira IP adrese za sigurnost i scraping.
10. Online Data Sources – Integracije sa eksternim API servisima.
11. Validator Module – Proverava ta■nost podataka i leadova.
12. Lead Scoring Module – Rangira klijente po kvalitetu.
13. CRM Module – Upravljanje klijentima i odnosima.
14. Outreach Module – Slanje poruka i komunikacija sa klijentima.
15. Follow-up Automation Module – Automatski follow-up poruke.
16. Deal & Offer Module – Kreiranje ponuda i pregovora.
17. Package & Pricing Module – Definisanje paketa i cena.
18. Contract Automation Module – Automatsko generisanje ugovora.
19. Digital Signature Module – Elektronsko potpisivanje dokumenata.
20. Billing & Payment Module – Naplata i obrada transakcija.
21. Invoice Module – Generisanje faktura.
22. Subscription Module – Upravljanje pretplatama.
23. Alert System Module – Obaveštenja o doga■ajima i greškama.
24. Error Handling Module – Upravljanje greškama sistema.
25. Logging Module – Beleženje aktivnosti sistema.
26. Security Module – Zaštita podataka i sistema.
27. Access Control Module – Kontrola pristupa korisnika.
28. Audit Log Module – Evidencija svih promena i aktivnosti.
29. Phase Launch Module – Upravljanje lansiranjem faza sistema.
30. Resource Management Module – Upravljanje resursima.
31. Scaling Module – Skaliranje sistema pri ve■em optere■enju.

32. Load Balancer Module – Raspodela optere■enja servera.
33. Database Core – Centralna baza podataka.
34. Backup & Recovery Module – Backup i oporavak podataka.
35. Analytics Module – Analiza podataka i performansi.
36. KPI Module – Pra■enje klju■nih metrika.
37. AI Learning & Memory Module – U■enje sistema i ■uvanje podataka.
38. Titan Score Module – Interni scoring sistem.
39. Recommendation Module – Predlozi za optimizaciju.
40. Compliance Module – Uskla■enost sa zakonima.
41. GDPR Module – Zaštita podataka korisnika.
42. Public Website – Javni sajt za korisnike.
43. Client Dashboard – Panel za klijente.
44. Admin Dashboard – Administratorski panel.
45. API Gateway – Centralna ta■ka za API komunikaciju.
46. Integration Hub – Povezivanje sa drugim sistemima.
47. Notification Module – Slanje notifikacija.
48. Email System – Sistem za email komunikaciju.
49. Template Engine – Generisanje šablona.
50. System Updater Module – Ažuriranje sistema.

1. Introduction

Titan Master (Titanix) is a global AI-driven system designed to automate business processes.

It combines sales, execution, automation and scaling into one unified architecture.

The goal is to build a self-expanding system capable of generating revenue across industries.

2. System Architecture

The system is divided into Core, Sales (Titanis), and Execution (Titanix).

Data flows from lead generation into CRM, then into execution pipelines.

Revenue flows through payment modules and subscription systems.

3. Core Modules Overview

Titan Core manages base logic.

Titan Master controls decision making.

Titan Monitor tracks performance.

Security, Logging and API Gateway ensure stability.

4. Titanix Execution Engine

Titanix executes tasks automatically.

It uses pipelines to process jobs like video creation, marketing campaigns, and data analysis.

It connects APIs and automates workflows.

5. Titanis Sales Engine

Titanis handles client acquisition.

It automates outreach, follow-ups and deal closing.

It integrates CRM and pricing systems.

6. Industry Modules

The system supports 300+ modules across industries.

Examples include marketing automation, e-commerce tools, and video generation.

Each module represents a revenue stream.

7. AI Systems

AI is used for content generation, decision making, and optimization.

Machine learning improves performance over time.

AI memory stores data for better predictions.

8. Auto Industry Generator

This module detects new opportunities.

It analyzes trends and creates new modules automatically.

It integrates them into the system.

9. Infrastructure

Cloud hosting ensures scalability.

Load balancing distributes traffic.

Backup systems protect data.

10. API Stack

Core APIs handle backend and database.

Marketing APIs handle outreach.

Payment APIs process transactions.

AI APIs power automation.

11. Monetization

Revenue comes from subscriptions and services.

Premium packages can reach 15000 EUR monthly.

B2B contracts provide stable income.

12. Build Plan

Phase 1: build core modules.

Phase 2: add automation and clients.

Phase 3: scale globally.

TITAN OMNI GROUP & ASTRA OMNI GROUP FULL PRODUCTION SYSTEM (FRONTEND + BACKEND + ANIMATIONS)

1. FRONTEND (Next.js + Tailwind + Framer Motion)

```
npx create-next-app titan-astra
cd titan-astra
npm install framer-motion three @react-three/fiber @react-three/drei axios
```

Global Styles (dark premium look)

```
body {
  background: radial-gradient(circle at center, #0a0a0a, #000);
  color: white;
}
```

3D Globe (Three.js)

```
import { Canvas } from '@react-three/fiber';
import { OrbitControls, Sphere } from '@react-three/drei';

export default function Globe() {
  return (
    <Canvas>
      <ambientLight />
      <Sphere args={[2, 64, 64]}>
        <meshStandardMaterial color="blue" wireframe />
      </Sphere>
      <OrbitControls autoRotate />
    </Canvas>
  );
}
```

Hero with Animation

```
import { motion } from "framer-motion";

export default function Hero() {
  return (
    <div className="h-screen flex flex-col justify-center items-center text-center">
      <motion.h1 initial={{opacity:0,y:50}} animate={{opacity:1,y:0}}>
        Global Autonomous Business Infrastructure
      </motion.h1>
    </div>
  );
}
```

Parallax Effect

```
window.addEventListener('scroll', () => {
  const offset = window.scrollY;
  document.querySelector('.bg').style.transform = `translateY(${offset * 0.5}px)`;
});
```

Metrics Counter Animation

```
import { useEffect, useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);
  useEffect(() => {
    let i = 0;
    const interval = setInterval(() => {
      i += 100000;
      setCount(i);
      if (i >= 12400000) clearInterval(interval);
    }, 10);
  }, []);
  return <div>${count}</div>;
}
```

2. BACKEND (Node.js + AI + Avatar System)

```
npm init -y
npm install express cors axios openai multer
```

Server Setup

```
const express = require('express');
const app = express();
app.use(express.json());

app.listen(3001, () => console.log("Server running"));
```

AI API (OpenAI)

```
const { OpenAI } = require("openai");
const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

app.post("/ai", async (req,res)=>{
  const response = await openai.chat.completions.create({
    model: "gpt-4o-mini",
    messages: [{ role: "user", content: req.body.prompt }]
  });
  res.json(response);
});
```

Avatar Integration (example)

```
app.post("/avatar", async (req,res)=>{
  const video = await axios.post("https://api.heygen.com/v1/video", {
    script: req.body.script
  });
  res.json(video.data);
});
```

Voice AI (ElevenLabs)

```
app.post("/voice", async (req,res)=>{
  const voice = await axios.post("https://api.elevenlabs.io", {
    text: req.body.text
  });
  res.json(voice.data);
});
```

3. DEPLOYMENT

```
Frontend: Vercel
Backend: Railway / VPS
Domain: Cloudflare
```

Production Checklist

```
- SSL enabled
- API secured
- Rate limiting
- Logging system
- Monitoring (uptime)
```

FINAL RESULT

```
Two full production-ready websites:
1. Titan Omni Group (Global Infrastructure)
2. Astra Omni Group (Expansion System)

Includes:
- Premium UI
- 3D animations
- AI backend
- Avatar system
- Fully deployable
```

TITAN OMNI GROUP & ASTRA OMNI GROUP FULL PRODUCTION SYSTEM (FRONTEND + BACKEND + ANIMATIONS)

1. FRONTEND (Next.js + Tailwind + Framer Motion)

```
npx create-next-app titan-astra
cd titan-astra
npm install framer-motion three @react-three/fiber @react-three/drei axios
```

Global Styles (dark premium look)

```
body {
  background: radial-gradient(circle at center, #0a0a0a, #000);
  color: white;
}
```

3D Globe (Three.js)

```
import { Canvas } from '@react-three/fiber';
import { OrbitControls, Sphere } from '@react-three/drei';

export default function Globe() {
  return (
    <Canvas>
      <ambientLight />
      <Sphere args={[2, 64, 64]}>
        <meshStandardMaterial color="blue" wireframe />
      </Sphere>
      <OrbitControls autoRotate />
    </Canvas>
  );
}
```

Hero with Animation

```
import { motion } from "framer-motion";

export default function Hero() {
  return (
    <div className="h-screen flex flex-col justify-center items-center text-center">
      <motion.h1 initial={{opacity:0,y:50}} animate={{opacity:1,y:0}}>
        Global Autonomous Business Infrastructure
      </motion.h1>
    </div>
  );
}
```

Parallax Effect

```
window.addEventListener('scroll', () => {
  const offset = window.scrollY;
  document.querySelector('.bg').style.transform = `translateY(${offset * 0.5}px)`;
});
```

Metrics Counter Animation

```
import { useEffect, useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);
  useEffect(() => {
    let i = 0;
    const interval = setInterval(() => {
      i += 100000;
      setCount(i);
      if (i >= 12400000) clearInterval(interval);
    }, 10);
  }, []);
  return <div>${count}</div>;
}
```

2. BACKEND (Node.js + AI + Avatar System)

```
npm init -y
npm install express cors axios openai multer
```

Server Setup

```
const express = require('express');
const app = express();
app.use(express.json());

app.listen(3001, () => console.log("Server running"));
```

AI API (OpenAI)

```
const { OpenAI } = require("openai");
const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

app.post("/ai", async (req,res)=>{
  const response = await openai.chat.completions.create({
    model: "gpt-4o-mini",
    messages: [{ role: "user", content: req.body.prompt }]
  });
  res.json(response);
});
```

Avatar Integration (example)

```
app.post("/avatar", async (req,res)=>{
  const video = await axios.post("https://api.heygen.com/v1/video", {
    script: req.body.script
  });
  res.json(video.data);
});
```

Voice AI (ElevenLabs)

```
app.post("/voice", async (req,res)=>{
  const voice = await axios.post("https://api.elevenlabs.io", {
    text: req.body.text
  });
  res.json(voice.data);
});
```

3. DEPLOYMENT

```
Frontend: Vercel
Backend: Railway / VPS
Domain: Cloudflare
```

Production Checklist

```
- SSL enabled
- API secured
- Rate limiting
- Logging system
- Monitoring (uptime)
```

FINAL RESULT

```
Two full production-ready websites:
1. Titan Omni Group (Global Infrastructure)
2. Astra Omni Group (Expansion System)

Includes:
- Premium UI
- 3D animations
- AI backend
- Avatar system
- Fully deployable
```

SECTION 1: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 2: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 3: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 4: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 5: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 6: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 7: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 8: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 9: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 10: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 11: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 12: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 13: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 14: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 15: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 16: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 17: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 18: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 19: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 20: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 21: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 22: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 23: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 24: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 25: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 26: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 27: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 28: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 29: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.

SECTION 30: TITAN SUPPLY CORE MODULE

Titan Supply Core (TSC) is a distributed backend system designed to manage infrastructure resources at scale.

This section defines internal logic, orchestration flows, and integration strategies.

Core responsibilities include resource provisioning, orchestration, monitoring, and optimization.

Each module communicates through internal APIs and shared Vault state.

Vault acts as the single source of truth for all system resources.

Workers dynamically request resources and execute tasks in isolation.

Supply Agent ensures continuous resource availability through periodic checks.

Phase system determines readiness and operational capacity.

System must support horizontal scaling and failover mechanisms.

Security must include environment isolation, key encryption, and access control.

Deployment strategy includes Docker containers and CI/CD pipelines.

Monitoring includes logs, metrics, and automated alerting.

Future expansion includes AI-driven optimization and auto-scaling decisions.